Learning Reusable Options by Decomposing Neural Policies

Parnian Behdin, Reza Abdollahzadeh, Kiarash Aghakasiri, and Levi H. S. Lelis
Department of Computing Science
University of Alberta
Alberta Machine Intelligence Institute (Amii)

Abstract

Options provide a natural form of behavioral abstraction in reinforcement learning, but discovering reusable options from trained neural policies remains difficult. A trained policy may contain many useful behaviors that can be reused in downstream problems, yet the number of candidate subpolicies that a neural network can encode grows exponentially with the number of hidden units. We propose a differentiable approach for extracting reusable options from neural policies. Our key observation is that for piecewise-linear neural networks, selecting a neural subprogram is equivalent to assigning each hidden unit one of three labels: inactive, active, or retained in the computation. This yields a mask space over neural subprograms that can be searched with gradient-based optimization. We further show that reusable options can require default input parameters: neuron masks determine what computation is reused, while input masks determine how that computation is called. Experiments in transfer settings with feedforward and recurrent policies show that the resulting options improve sample efficiency on downstream learning.

1 Introduction

In transfer learning settings, an agent benefits from reusing skills learned in previous tasks. Options provide a natural representation for this kind of reuse: instead of choosing only primitive actions, the agent can also choose temporally extended actions that execute useful behaviors over multiple time steps [Sutton et al., 1999]. Portable options can make downstream learning more sample efficient because they let the agent reason with behaviors instead of individual actions [Konidaris and Barto, 2007]. The difficulty is that useful options are not usually given to the agent.

In this paper we study how to discover reusable options from trained neural policies. A neural policy that solved a previous task may contain useful behaviors for solving a new task, but reusing the entire policy is often too coarse. The policy may depend on features that are specific to the task in which it was trained. At the same time, the behavior we would like to reuse may be only a small part of the computation performed by a network encoding the policy. This creates a decomposition problem: given a trained neural policy, we would like to extract the parts of the policy that encode reusable behaviors and turn them into options for downstream learning.

This problem is intuitive for programmatic representations. If a solution is written as a program, then reuse can be implemented by extracting functions from previous solutions and storing them in a library. Library-learning methods compress solutions into reusable abstractions that can reduce the complexity of solving future tasks [Ellis et al., 2023, Cao et al., 2023, Bowers et al., 2023, Rahman et al., 2024]. Neural policies are also programs in the sense that they compute actions through a composition of elementary operations, but their reusable components are not explicitly named functions. They are entangled with the rest of the network and are not necessarily easy to extract.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

Prior work showed that ReLU neural policies can be decomposed into neural trees, whose subtrees correspond to subprograms of the original policy [Alikhasi and Lelis, 2024]. These subprograms can be used as options. This provides a direct connection between neural policy decomposition and option discovery. However, the approach requires enumerating candidate subtrees of the neural tree. The number of candidates grows exponentially with the number of hidden units, which makes exhaustive decomposition infeasible beyond very small networks. As a result, the main challenge is how to search the large space of possible options of trained policies without enumerating them.

We address this challenge by turning neural decomposition into a differentiable masking problem. Our key insight is that extracting a subtree from a neural tree can be represented by assigning a label to each hidden ReLU unit of the original network. A hidden unit can be removed from the extracted computation or kept as part of the computation being reused. Thus, searching over neural subprograms can be implemented as learning masks over hidden units. The discrete masks define the extracted option, while continuous mask parameters allow us to search this space with gradient descent. We call our method DIDEC, for "Differentiable Decomposition".

The decomposition of a policy is not enough to define a reusable option. A reusable behavior also needs to specify how it is called in a new context. Consider a behavior that was learned in one location of an environment but should be reused in another location. The computation encoding the behavior may still depend on location features that were present when the behavior was originally learned. In this case, selecting the right hidden units is not sufficient; the extracted computation may also need default input values that allow it to behave as if it were called in the context where the behavior was learned. DIDEC therefore learns input masks together with hidden-unit masks. Neuron masks determine the subprogram to execute; input masks specify how to call such a subprogram. The learned masks determine reusable neural subprograms. Each subprogram defines the policy of an option, while a simple deterministic termination rule determines how long the option is executed.

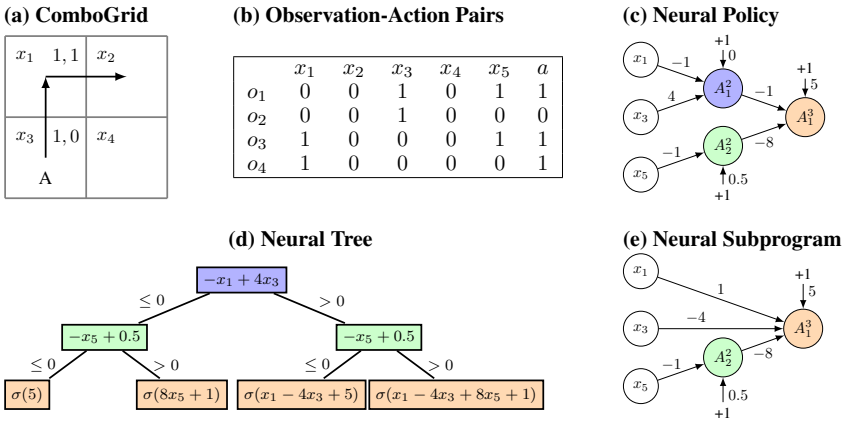
We evaluate DIDEC in transfer learning settings with feedforward and recurrent policies. The policies are trained on previous tasks and decomposed into options used by an agent learning a downstream task. Our experiments use domains with both complete and partial observability. The networks we decompose induce option spaces far beyond what enumeration can achieve, yet the options extracted by DIDEC improve sample efficiency compared to baselines that reuse previous policies without decomposition. These results support the hypothesis that trained neural policies can contain reusable behavioral abstractions, and that differentiable neural decomposition can discover these abstractions.

2 Problem Formulation

We consider a transfer learning setting in which an agent has solved a sequence of POMDPs and is evaluated on a new POMDP. We write the j-th POMDP as $P_j = (S_j, A, O, p_j, q_j, r_j, S_{0,j})$, where S_j is the state space, A is the action space, O is the observation space, p_j is the transition function, q_j is the observation function, r_j is the reward function, and $S_{0,j}$ is the initial-state distribution. We assume that the POMDPs share the same action space and observation representation, so that policies trained on previous tasks can be evaluated on observations from other tasks. We use the POMDP formalism to encompass the fully and partially observable environments we experiment with.

Before solving a target POMDP P_i, the agent has access to trained neural policies $\Pi_{\text{train}} = \{\pi_1, \ldots, \pi_{i-1}\}$ and observation-action trajectories $\mathcal{T}_{\text{train}} = \{T_1, \ldots, T_{i-1}\}$ generated by rolling out these policies in their corresponding POMDPs. Each trajectory is a sequence $T_j = \{(o_0, a_0), (o_1, a_1), \ldots, (o_{T_j}, a_{T_j})\}$. Our goal is to use Π_{train} and $\mathcal{T}_{\text{train}}$ to construct a set of reusable options Ω that improves the sample efficiency of learning in the downstream task P_i.

An option is a tuple $\omega = (I_\omega, \pi_\omega, T_\omega)$, where $I_\omega \subseteq O$ is the set of observations in which the option can be invoked, π_ω is the option policy, and T_ω is the termination function [Sutton et al., 1999]. We use call-and-return execution: once an option is selected, the agent follows π_ω until the option terminates. In this work, extracted options are intended to be portable across the target observation space, so we set $I_\omega = O$. We also use deterministic fixed-duration termination: an extracted option runs for a fixed number of primitive steps. Thus, DIDEC learns π_ω, and the fixed termination rule turns the resulting masked neural computation into a temporally extended action.



	x_1	x_2	x_3	x_4	x_5	a
o_1	0	0	1	0	1	1
o_2	0	0	1	0	0	0
o_3	1	0	0	0	1	1
o_4	1	0	0	0	0	1

Figure 1: **(a)** Instance of a ComboGrid environment with combos of length two. The sequence $1, 0$ moves up, while $1, 1$ moves right. Agent 'A' starts at x_3 and reaches x_2 after $1, 0, 1, 1$. **(b)** Observation-action pairs for the trajectory in (a). The observation includes the agent location and bit x_5, set to 1 if no action was taken in a sequence, and 0 otherwise. **(c)** Neural network that fits the data in (b), with ReLU hidden units and a sigmoid output; zero-weight connections are omitted. **(d)** Neural tree equivalent to (c): the left subtree encodes the "right combo" and the right subtree with default $x_3 = 1$ encodes the "up combo". **(e)** Neural subprogram obtained by setting A_1^2 active.

Given trained neural policies Π_{train} and trajectories $\mathcal{T}_{\text{train}}$ from previously solved POMDPs, our task is to discover a set of options Ω such that an agent learning P_i with action space augmented from A to $A \cup \Omega$ requires fewer interactions to achieve high return than an agent learning with A alone.

3 Neural Policy Decomposition

We first describe how we view a trained neural policy as an object that can be decomposed into candidate option policies. This section focuses on feedforward policies with piecewise-linear activations, in which the connection among neural networks, neural trees, and masks is direct.

3.1 Feedforward Neural Policies

We assume that the policies in Π_{train} are encoded in fully connected feedforward neural networks, such as the one shown in Figure 1 (c). Each layer k has n_k neurons $(1, \ldots, n_k)$, with $n_1 = |X|$, where X is the observation vector passed as input to the network. The trainable parameters are the weights and biases between consecutive layers. We denote the weights between layers k and $k+1$ by $W^k \in \mathbb{R}^{n_{k+1} \times n_k}$ and the biases by $B^k \in \mathbb{R}^{n_{k+1}}$. The r-th row of W^k, denoted W_r^k, with the r-th entry of B^k, denoted B_r^k, represent the weights and bias of the r-th neuron of the $(k+1)$-th layer.

We denote by $A^k \in \mathbb{R}^{n_k}$ the vector of values produced in the k-th layer. Here, $A^1 = X$, and A^m is the output of a network with m layers. The pre-activation values of layer k are $Z^k = W^{k-1}A^{k-1} + B^{k-1}$, and the layer output is $A^k = g(Z^k)$, where $g(\cdot)$ is the activation function. For intermediate layers, we consider piecewise-linear activation functions, such as ReLU, where $g(z) = \max(0, z)$. The output layer produces the action logits or action probabilities defining the policy.

3.2 Neural Trees and Neural subprograms

A feedforward network with ReLU activations maps to an equivalent neural tree [Alikhasi and Lelis, 2024]. We explain this mapping with the example in Figure 1. The example shows a small ComboGrid environment (a), a trajectory of observation-action pairs (b), a neural policy that fits this trajectory (c), the neural tree equivalent to this policy (d), and a compressed policy obtained from one masked computation (e). The default input value shown in the figure previews the input-mask mechanism introduced in Section 4; here, we focus on the neural-tree and hidden-mask correspondence.

Each hidden ReLU neuron is mapped to a level in the neural tree. The order in which hidden neurons are assigned to levels is arbitrary. Different orderings produce different neural trees that represent the same neural network and compute the same function. In Figure 1, neuron A_1^2 is represented by the root level of the tree, A_2^2 by the second level of the tree, and A_1^3 by the tree's leaves. Each internal node considers the two possible outcomes of a ReLU neuron: if $Z \leq 0$, the output is 0 and the computation follows the left branch; if $Z > 0$, the output is Z and the computation follows the right branch. A path from the root to a leaf corresponds to an activation pattern of the network [Montúfar et al., 2014]: the active/inactive state of each neuron for a given input.

Example 1. *We trace two root-to-leaf paths in Figure 1(d). Each internal node branches on the sign of the pre-activation value of one hidden unit: the left branch fixes its output to* 0 *(inactive), and the right branch fixes its output to its pre-activation value (active).*

Left–right path. *Take the left branch at the root (A_1^2 inactive, so $A_1^2 = 0$) and the right branch at the second level (A_2^2 active, so $A_2^2 = Z_2^2 = -x_5 + 0.5$). The output is*

$$A_1^3 = \sigma\big(-A_1^2 - 8A_2^2 + 5\big) = \sigma(-0 - 8(-x_5 + 0.5) + 5) = \sigma(8x_5 + 1),$$

which is the second leaf of Figure 1(d).

Right–left path. *Take the right branch at the root (A_1^2 active, so $A_1^2 = Z_1^2 = -x_1 + 4x_3$) and the left branch at the second level (A_2^2 inactive, so $A_2^2 = 0$). The output is*

$$A_1^3 = \sigma(-(-x_1 + 4x_3) - 8 \cdot 0 + 5) = \sigma(x_1 - 4x_3 + 5),$$

which is the third leaf of Figure 1(d). The remaining two leaves are obtained analogously.

Each subtree of the neural tree provides a subprogram of the policy. In the options view, such a subprogram is a candidate option policy: it maps observations to action probabilities, but it does not by itself specify how long the behavior should be executed. Section 4 describes how DIDEC learns such subprograms from data and wraps them with deterministic termination to produce options.

3.3 Subtrees as Hidden-Unit Masks

Although the neural tree depends on the order in which we branch on hidden units, the subprogram represented by a subtree depends only on which units are fixed to 0, which are fixed to their pre-activation value, and which remain inside the subtree. This observation lets us replace subtrees with a simpler object, which we explain next. Let $H = \{h_1, \ldots, h_d\}$ be the hidden ReLU units of a feedforward policy π_θ. A hidden-unit mask is a function assigning a label to a unit:

$$m : H \to \{\mathrm{inactive}, \mathrm{active}, \mathrm{retained}\},$$

where $\mathrm{inactive}$ replaces h's output with 0, active replaces it with its pre-activation value, and $\mathrm{retained}$ keeps the original ReLU computation. We write $\mathcal{M}_H(\pi_\theta)$ for the set of all such masks. Since each neuron can be assigned one of three labels, $|\mathcal{M}_H(\pi_\theta)| = 3^{|H|}$.

Searching in the space of hidden-unit masks allows us to search over all possible subtrees of all neural trees representing the original network. To see this, consider any subtree of any neural tree representing the policy. The neurons on the path from the root to the subtree are fixed as inactive or active, according to the branches selected, while the neurons inside the subtree are retained. These choices define a mask that computes the same function as the subtree. Therefore, every subtree of every neural tree representing the original network can be represented by a hidden-unit mask.

Different neural trees may have subtrees that make the same neuron-level choices: inactive, active, or retained. These subtrees are computationally equivalent. The order in which a neural tree branches on retained hidden units does not affect what is computed, provided that neurons in earlier network layers appear at shallower levels of the tree than neurons in later layers. Within each layer, the ordering is arbitrary. Likewise, the order in which neurons are fixed as active or inactive is unimportant; only their assignments matter. Thus, subtrees that make the same neuron-level choices compute the same function and belong to the same equivalence class. This means that, by searching in the space of masks, we search in the space of equivalent classes of subtrees of all neural trees of the network.

This correspondence states what masks can represent, not what gradient descent will find. If a behavior is implemented by a subtree of a neural tree representing π_θ, then a mask that recovers it exists. DIDEC does not guarantee that the gradient optimization we describe in Section 4 finds

such a mask. Moreover, a behavior that is not encoded in π_θ may not be recovered by masking it, just as we cannot expect to extract a function from a program that does not contain it. The correspondence between masks and equivalence classes of neural subtrees justifies our mask-based approach: searching over masks covers all equivalent classes of all neural trees representing π_θ, and learning continuous mask parameters gives a differentiable relaxation of this discrete search.

3.4 Masked Forward Pass

We represent the masks of the neurons in layer k by a matrix $\Theta^k \in \mathbb{R}^{n_k \times 3}$. The parameters in the i-th row of Θ^k define a distribution over three possible states for neuron i: inactive, active, or retained in the extracted computation. We compute the following, where $M^k_{i,:} \in \{0,1\}^3$.

$$Z^k = W^{k-1}A^{k-1} + B^{k-1} \quad \text{and} \quad M^k_{i,:} = \mathrm{OneHot}\big(\mathrm{Softmax}(\Theta^k_{i,:})\big)\,,$$

The masked layer output is then

$$A^k = M^k_{:,1} \odot \mathbf{0} + M^k_{:,2} \odot Z^k + M^k_{:,3} \odot \mathrm{ReLU}(Z^k).$$

Rows with a 1 in the first column set the corresponding neuron to 0 and make it inactive; rows with a 1 in the second column replace the ReLU output by the pre-activation value Z^k and make the neuron active; rows with a 1 in the third column keep the original ReLU and retain the neuron's computation.

The function OneHot is non-differentiable because of its implicit maximum. When updating Θ^k with gradient descent, we use the straight-through estimator [Bengio et al., 2013], which passes the gradient computed up to the maximum operation in the backward pass directly to the Softmax layer. We also considered the modified $\tanh$ function proposed by Pitis [2017] and Koul et al. [2019] for discretizing into three values, but preliminary experiments favored the Softmax approach.

Example 2. *The subprogram shown in Figure 1(e) is obtained from the network in Figure 1 (c) by setting A^2_1 active and retaining A^2_2. The subprogram is obtained with the following neuron mask*

$$M^2 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix},$$

where row 1 corresponds to A^2_1 (active, column 2) and row 2 to A^2_2 (retained, column 3). For an observation X, the pre-activation vector is $Z^2 = W^1X + B^1$, and the masked layer output is

$$A^2 = M^2_{:,1} \odot \mathbf{0} + M^2_{:,2} \odot Z^2 + M^2_{:,3} \odot \mathrm{ReLU}(Z^2) = \begin{pmatrix} Z^2_1 \\ \mathrm{ReLU}(Z^2_2) \end{pmatrix} = \begin{pmatrix} -x_1 + 4x_3 \\ \mathrm{ReLU}(-x_5 + 0.5) \end{pmatrix}.$$

Composing with the output layer gives $A^3_1 = \sigma(-A^2_1 - 8A^2_2 + 5) = \sigma(x_1 - 4x_3 - 8A^2_2 + 5)$, which is the subprogram shown in Figure 1(e). Because A^2_1 is active, its ReLU is no longer needed and the unit can be folded into the output neuron, leaving a network with a single hidden unit.

4 DIDEC: Learning Options from Neural Policy Decompositions

Section 3 defines the mask space induced by the neural-tree decomposition of a feedforward ReLU policy. We now describe how DIDEC learns masks from trajectories generated by previously trained policies and turns the resulting masked subprograms into options. Throughout this section, the weights of the trained policies are fixed. The only learned parameters are the masks.

4.1 From Masked Subprograms to Options

A hidden-unit mask selects a subprogram of a trained policy. Such a subprogram maps observations to action probabilities, so it is the policy of an option. To obtain a temporally extended action, we wrap the masked policy in a repeat loop. If π^Θ_θ denotes the policy obtained by applying mask parameters Θ to a source policy π_θ, then a length-b option has the form $\mathrm{repeat}(b) : \pi^\Theta_\theta$. When this option is selected, the agent executes π^Θ_θ for b primitive steps and then returns control to the high-level policy.

This construction follows the options view of neural decomposition: the extracted subprogram is not transferred as an entire solution to a previous task, but as a reusable behavior that can be invoked by a downstream learner. The next sections describe the parameters that define such a behavior.

4.2 Input Masks as Default Parameters

A reusable option is not specified only by its computation. It also needs to specify how this computation is called in a new context. A subprogram extracted from a policy trained in one environment might depend on input features that describe the context in which the behavior was learned. If those features are read directly from a new environment, the same computation may no longer produce the behavior. In this case, the extracted option needs default input parameters.

DIDEC learns these default parameters by masking the input observation. For binary inputs, we use the same Softmax and one-hot discretization used for hidden-unit masks. Each input coordinate is assigned one of three labels: read the value from the environment, set it to 0, or set it to 1. Let $X \in \{0,1\}^{n_1}$ be the observation and $\Theta^X \in \mathbb{R}^{n_1 \times 3}$ be the input-mask parameters. For each input i,

$$M^X_{i,:} = \mathrm{OneHot}(\mathrm{Softmax}(\Theta^X_{i,:})) \in \{0,1\}^3,$$

with one entry for each possible label. The masked input is then $\widetilde{X}_i = M^X_{i,1} \cdot X_i + M^X_{i,2} \cdot 0 + M^X_{i,3} \cdot 1$.

Example 3. *In this example we combine the neuron mask of Example 2 with an input mask that defaults x_3 to 1 and reads the remaining inputs from the environment. The input mask has*

$$M^X_{3,:} = (0,\,0,\,1) \quad \textit{(default to 1)}, \qquad M^X_{i,:} = (1,\,0,\,0) \quad \textit{for } i \in \{1,2,4,5\} \textit{ (read from env.)},$$

so that $\widetilde{X} = (x_1,\,x_2,\,1,\,x_4,\,x_5)$. The masked policy from Example 2, evaluated on $\widetilde{X}$, computes

$$\pi^{\Theta}_{\theta}(\widetilde{X}) \;=\; \sigma\big(x_1 - 4 - 8\,\mathrm{ReLU}(-x_5 + 0.5) + 5\big).$$

Suppose the agent is at location x_1 in a new environment, so the actual observation has $x_1 = 1$ and $x_3 = 0$, and consider the second step of the up combo, where $x_5 = 0$. Then $\mathrm{ReLU}(-x_5 + 0.5) = 0.5$. Without the input mask, the masked policy (using $x_3 = 0$ instead of the default 1) evaluates to $\sigma(1 - 0 - 4 + 5) = \sigma(2) \approx 0.88$, predicting action 1 rather than the action 0 required by the up combo. With the input mask, $\sigma(1 - 4 - 4 + 5) = \sigma(-2) \approx 0.12$, correctly predicting action 0. Setting $x_3 = 1$ lets the agent "pretend" it is in x_3, so the up combo generalizes to other locations.

For continuous inputs, the same idea can be implemented by learning whether each coordinate should be read from the environment or replaced by a learned default value. In this case, each input mask has two labels. Let $\Theta^X \in \mathbb{R}^{n_1 \times 2}$ be the mask parameters and let $\Theta^D \in \mathbb{R}^{n_1}$ be the parameters of the default input values. We compute the following, where g is chosen to match the range of the input coordinate. For example, if the input values are in $[-1, 1]$, one can use $g = \tanh$.

$$M^X_{i,:} = \mathrm{OneHot}(\mathrm{Softmax}(\Theta^X_{i,:})) \in \{0,1\}^2 \quad \text{and} \quad \widetilde{X}_i = M^X_{i,1} \cdot X_i + M^X_{i,2} \cdot g(\Theta^D_i),$$

Although input masking can sometimes be sufficient to obtain a reusable option, hidden-unit masking has an additional benefit: it compresses the extracted policy. A hidden unit masked as inactive can be removed by substituting its output with zero, and a hidden unit masked as active can be removed by substituting its output with its pre-activation value. The compressed policy in Figure 1 (e) shows this effect. Once neuron A^2_1 is masked as active, its ReLU is no longer needed, and the output neuron can be rewritten directly as a function of the inputs and A^2_2, without A^2_1, namely $A^3_1 = \sigma(x_1 - 4x_3 - 8A^2_2 + 5)$; A^2_1 can be removed from the extracted model. More generally, every neuron at layer k that is masked as active or inactive can be removed from the model by rewriting the affine functions of neurons in layer $k+1$. In this sense, neuron masking not only selects reusable option policies; it also produces smaller policies by removing computation that is fixed by the extracted subprogram. To illustrate the compression effect of neuron masking on the actor network of an agent, in our experiments with MultiRoom, masking both inputs and neurons reduces the number of actor parameters from 52,486 to 7,730 on average, a reduction of 85.3%.

4.3 Recurrent Option Policies

In partially observable problems, policies are often represented by recurrent models such as LSTMs [Hochreiter and Schmidhuber, 1997] and GRUs [Chung et al., 2014, Bahdanau et al., 2015]. When extracting options from recurrent policies, neuron and/or input masks are insufficient. We must also specify the recurrent state from which the option starts. Recurrent policies can be viewed as approximating finite-state machines, where the hidden state represents the current mode of the underlying machine. An option extracted from a recurrent policy therefore needs an initial mode.

Algorithm 1 DIDEC's Neural Decomposition with Behavior Cloning

Require: Policies Π_{train}, trajectories $\mathcal{T}_{\text{train}}$, maximum length $z_{\max}$, epochs E, learning rate α
Ensure: Candidate option set Ω_{cand}

```
Ω_cand ← ∅
for each sub-trajectory τ of length 2, ..., z_max in T_train do
    Let j be the index of the policy that generated τ
    for each source policy π_ℓ ∈ Π_train with ℓ ≠ j do
        Initialize mask/default parameters Θ
        Train Θ for E steps to minimize −Σ_{(o,a)∈τ} log π_ℓ^Θ(a | o)
        Ω_cand ← Ω_cand ∪ {repeat(|τ|) : π_ℓ^Θ}
return Ω_cand
```

DIDEC learns this initial mode together with the input masks. For the initial hidden vector $h_0 \in [-1, 1]^r$, DIDEC learns parameters Θ^h and sets $h_0 = \tanh(\Theta^h)$, where Θ^h has the same shape as h_0. For LSTMs, we also learn the initial memory vector c_0. Since c_0 is unbounded, we parameterize it with two tensors Θ^s and Θ^m, representing sign and magnitude $c_0 = \tanh(\Theta^s) \odot \text{SoftPlus}(\Theta^m)$.

When the option is invoked, the recurrent policy is initialized with the learned hidden and memory vectors and executed for its duration. In this way, learning the state selects the option's initial mode, while input masks control which observations are allowed to define the behavior from that mode.

4.4 Learning Masked Option Policies with Behavior Cloning

We use the trajectories generated by the policies in Π_{train} to learn masks by behavior cloning [Schaal, 1997]. Let $\mathcal{T}_{\text{train}} = \{T_1, \ldots, T_{i-1}\}$ be the trajectories obtained by rolling out the trained policies in their corresponding POMDPs. For each trajectory T_j, DIDEC considers sub-trajectories of observation-action pairs with length $z \in \{2, 3, \ldots, z_{\max}\}$, where $z_{\max}$ is a hyperparameter.

Each sub-trajectory is used as a short behavioral specification. Given a sub-trajectory

$$\tau = ((o_t, a_t), (o_{t+1}, a_{t+1}), \ldots, (o_{t+z-1}, a_{t+z-1}))$$

from trajectory T_j, we learn masks for a source policy $\pi_\ell \in \Pi_{\text{train}}$ with $\ell \neq j$. The condition $\ell \neq j$ prevents the method from simply recovering the policy that generated the sub-trajectory. Instead, it asks whether a different trained policy contains a subprogram that can reproduce the behavior in τ, which encourages the extraction of behaviors that are reusable across tasks.

For a fixed source policy π_ℓ and sub-trajectory τ, DIDEC minimizes the log loss $-\sum_{(o,a)\in\tau} \log \pi_\ell^\Theta(a \mid o)$, where π_ℓ^Θ denotes the source policy with the current input masks, hidden-unit masks, and recurrent-state parameters, when applicable. The weights of π_ℓ are frozen; gradient descent updates only Θ. After training, the masked policy π_ℓ^Θ is wrapped as an option of length $|\tau|$. Longer sub-trajectories generate longer options, but they also tend to be more specialized to the source task. The hyperparameter $z_{\max}$ therefore controls the trade-off between the number and duration of generated options and the risk of extracting overly specific behaviors.

4.5 Candidate Option Extraction

Algorithm 1 summarizes the option-extraction procedure. The algorithm returns a candidate option set Ω_{cand}. In our experiments, this candidate set is pruned before downstream learning so that the high-level agent does not need to choose among too many options. We use the subset selection process of Alikhasi and Lelis [2024], which we describe in Appendix J for the interested reader.

Let $T_{\max}$ be the length of the longest trajectory in $\mathcal{T}_{\text{train}}$. For each trajectory, DIDEC considers $O(T_{\max} z_{\max})$ sub-trajectories. Each sub-trajectory is paired with each source policy that did not generate it. Thus, since $N = |\mathcal{T}_{\text{train}}| = |\Pi_{\text{train}}|$, the number of mask-learning problems is $O(T_{\max} z_{\max} N^2)$. If each mask-learning problem uses E gradient steps and the source policy has W parameters, the cost of generating Ω_{cand} is $O(T_{\max} z_{\max} N^2 E W)$. The important point is that this cost does not include the exponential number of neural-tree subprograms. DIDEC searches this space by optimizing mask parameters rather than enumerating candidate subtrees.

5 Related Work

Options Early work relied on manually designed options [Sutton et al., 1999], but many methods now learn options automatically—though they often require human choices such as the number of options [Bacon et al., 2017, Igl et al., 2020] or their duration [Frans et al., 2017, Tessler et al., 2017]. DIDEC avoids such supervision but depends on data from previously solved tasks. While options have also been explored for improving exploration [Jinnai et al., 2020, Machado et al., 2023] and faster credit assignment [Mann and Mannor, 2014], DIDEC focuses on extracting functions that encode reusable behaviors to facilitate downstream knowledge transfer [Konidaris and Barto, 2007].

Transfer Learning Knowledge transfer across tasks has been studied via regularization [Kirkpatrick et al., 2017], architectural priors [Rusu et al., 2016b, Yoon et al., 2017, Schwarz et al., 2018], and experience replay [Rolnick et al., 2019]. A common strategy is to reuse parts of pretrained models [Clegg et al., 2017, Shao et al., 2018]. Unlike these approaches, DIDEC transfers knowledge by extracting reusable sub-programs (options) through gradient-based network decomposition.

Policy Distillation and Behavior Priors Policy distillation transfers the behavior of a trained policy into a new, often much smaller, network [Rusu et al., 2016a]. DIDEC also produces smaller policies, since neurons masked as active or inactive can be removed (Section 4.2). However, DIDEC does not fit a new network; it searches for subprograms of an existing one. Other methods use demonstrations or trained policies as behavioral priors, either by regularizing the learner toward the demonstrated behavior [Nair et al., 2018, Lu et al., 2023] or by learning a prior that shapes exploration [Singh et al., 2020]. We compare DIDEC with a distillation and a behavior prior baseline in Appendix D.

Library Learning DIDEC is a library-learning method [Cao et al., 2023, Bowers et al., 2023, Rahman et al., 2024, Palmarini et al., 2024]. As a representative example, DreamCoder [Ellis et al., 2023] builds a library of reusable lambda calculus functions to solve supervised learning problems. Similarly, DIDEC constructs a library of reusable programs, but the programs are extracted from neural networks and the underlying problem is reinforcement learning, and not supervised learning. DIDEC contributes toward bridging symbolic and neural paradigms in library-learning methods.

Masking Networks Masking has been used in transfer learning, e.g., SupSup [Wortsman et al., 2020] and Modulating Masks [Ben-Iwhiwhu et al., 2022], but these approaches mask weights, not neurons or input features. In contrast, DIDEC masks neurons with piecewise-linear activations, allowing for the extraction of subprograms. Input masking has also been explored, for mitigating visual distractions [Bertoin et al., 2022, Grooten et al., 2024] or learning auxiliary tasks [Yu et al., 2022], but DIDEC uses input masking to define default parameters to enable generalization.

6 Empirical Evaluation

We hypothesize that DIDEC can extract options that generalize to downstream tasks. To evaluate our hypothesis, we use Proximal Policy Optimization (PPO) [Schulman et al., 2017] with a feedforward network or an LSTM for the policy. We use an LSTM in ComboGrid, where the agent must remember previous actions. Our network sizes range from 64 to 256 neurons, resulting in a range of 3.43×10^{30} to 1.39×10^{122} subtrees, which far exceeds the capacity of enumerative approaches. We detail the architectures in Appendix E.

Our experiments are on MiniGrid [Chevalier-Boisvert et al., 2023], MiniHack's Corridor [Samvelyan et al., 2021], ComboGrid [Alikhasi and Lelis, 2024], and Meta-World's "Push the Puck" [Yu et al., 2021]. We use PPO in all of our experiments. Neuron masking can be used when the activation functions are piecewise linear, and input masking can be used when the input domains are finite, which includes continuous inputs, as in Meta-World. We use feedforward networks with MiniGrid, MiniHack, and Meta-World, and an LSTM with ComboGrid. A complete description of the environments used is given in Appendix K.

Alikhasi and Lelis [2024] compared an enumerative version of DIDEC to a range of option-learning methods, including Modulating Masks [Ben-Iwhiwhu et al., 2022], Progressive Neural Networks [Rusu et al., 2016b], Option-Critic [Bacon et al., 2017], ez-greedy [Dabney et al., 2021], and DCEO [Klissarov and Machado, 2023], and showed a clear advantage for the decomposition approach. That advantage, shared by DIDEC, arises from leveraging policies trained on $\{P_j\}_{j=1}^{i-1}$ to improve sample efficiency on P_i, which previous methods were not designed to leverage.

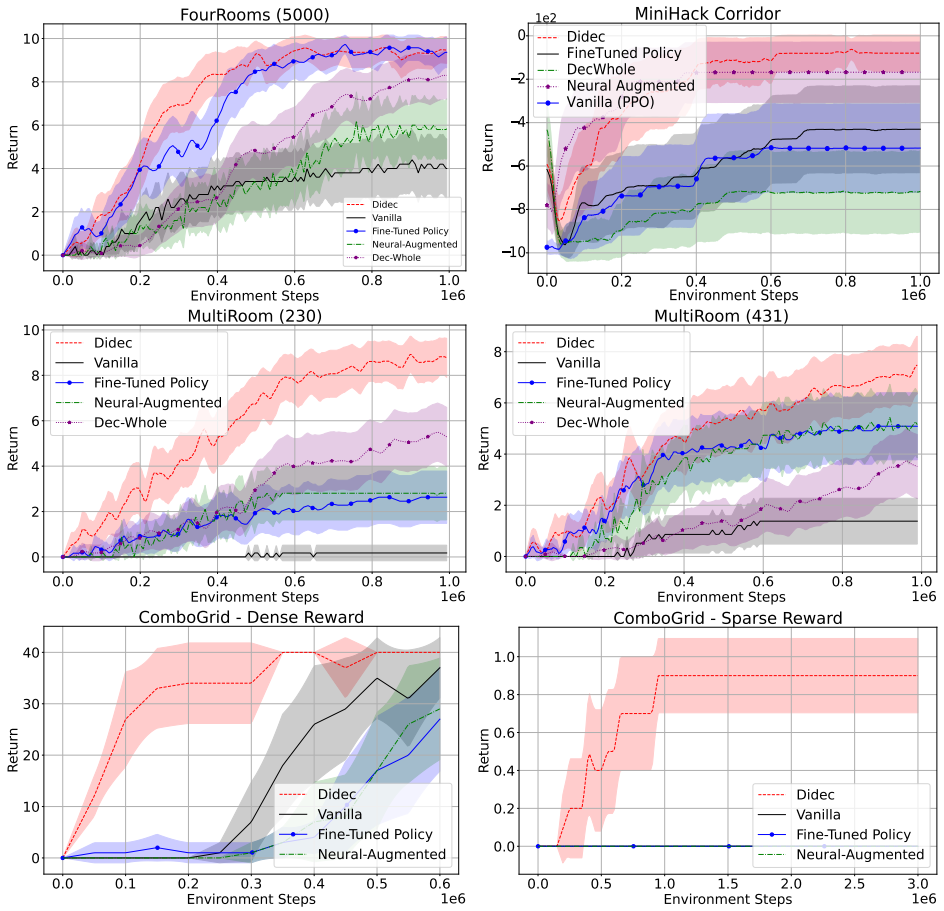


Figure 2: Learning curves of DIDEC and baselines on downstream tasks with temporally extended options. Curves show average return with 95% confidence intervals, using 50 seeds for FourRooms, 57 for MultiRoom, 10 for ComboGrid, and 22 for MiniHack.

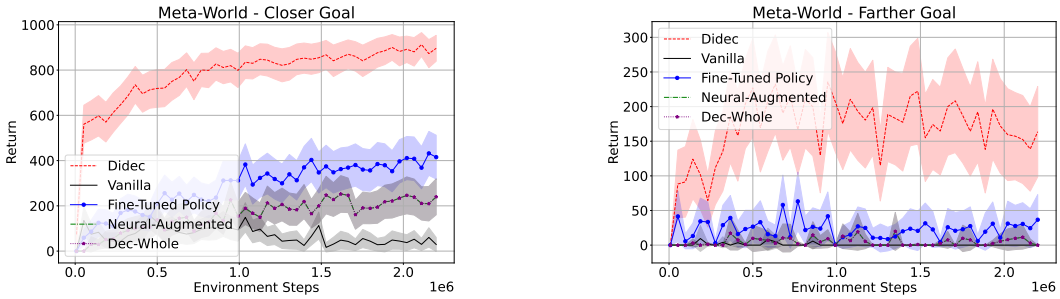


Figure 3: Meta-World results: average return with 95% confidence intervals over 97 seeds.

To evaluate DIDEC, we compare it against baselines that also exploit previously learned policies. The first augments the action space with one-step options derived from earlier policies (**Neural-Augmented**). The second is like DIDEC except it reuses prior policies as a single undivided function rather than decomposing them (**Dec-Whole**). The third is also like DIDEC except that it fine-tunes the policy weights instead of training masks (**Fine-Tuned Policy**). Finally, we use the learning algorithm of each experiment (PPO) with the primitive action space only, without options (**Vanilla**).

These baselines also serve as an ablation study for DIDEC. Neural-Augmented does not consider neural decomposition and temporal abstraction; Dec-Whole does not consider neural decomposition;

Fine-Tuned Policy trains the neural network weights rather than the mask weights. Together, these baselines evaluate different components of DIDEC. Further ablations are in Appendices C and G.

For each option-based approach, we augment the agent's action space with the set of options that each method selects. Then, we train a policy on the test problem of each domain, an action-augmented agent with PPO. In Appendix F, we present the values considered in our hyperparameter sweep for DIDEC and baselines. We considered masking neurons, input features, or both, as hyperparameters of DIDEC. Our empirical results are presented in plots, where the y-axis represents the average return and the x-axis represents the number of agent interactions.

Figure 2 and 3 present the results. The results support our hypothesis: DIDEC is competitive across all domains and substantially more sample-efficient in several of them. The performance of DIDEC on MultiRoom (230) is more evident than on MultiRoom (431) and MiniHack. We conjecture that this occurs because one of the rooms in problem 230 is much narrower than those in the training problems (see Appendix K), making it more difficult to generalize. The structures encountered in the training of MultiRoom and MiniHack are more similar to those found in the test problem, thus allowing the baselines also to discover options that generalize. DIDEC also presents similar advantages over the tested baselines on the continuous Meta-World tasks, where baselines fail to learn any meaningful policies. The results on ComboGrid (dense and sparse) demonstrate that DIDEC is also applicable to recurrent models. In this domain, Dec-Whole did not select any options, so it behaves like Vanilla.

The options DIDEC extracts differ across domains. In ComboGrid, the options ignore the observation and repeat combos (Appendix B); they are closer to macro-actions than to observation-dependent behaviors. The agent was not instructed to learn such options; DIDEC learned from data that open-loop combos transfer well in this domain. In MiniGrid, the options read part of the input and their behavior depends on the observation; for example, an option that picks up an object does so from different starting locations. MiniGrid's egocentric view makes transfer easier, but it does not guarantee it: baselines transfer from SimpleCrossing to FourRooms, where walls are straight in both training and test, but not from Unlock to MultiRoom, where the test problem has jagged walls.

Appendix D compares DIDEC with FAMP [Kuric and van Hoof, 2023], policy distillation, and behavior priors: DIDEC outperforms FAMP and distillation, while behavior priors learn faster on MultiRoom, but fail on ComboGrid. Appendix B shows that some of the options DIDEC learns are interpretable, while Appendix H shows how DIDEC's learned options can help with exploration in downstream tasks. Finally, Appendix I presents a running time analysis of DIDEC.

7 Conclusions and Limitations

We introduced DIDEC, a differentiable method for discovering reusable options from trained neural policies. DIDEC replaces exhaustive enumeration of neural-tree subprograms with mask optimization: hidden-unit masks select reusable computation, and input-default masks specify how that computation is invoked in new contexts. This formulation connects neural policy decomposition to option discovery. Experiments with feedforward and recurrent policies show improved downstream sample efficiency, while Meta-World suggests that the same masking idea can also be useful in continuous-control settings. These results support the view that trained neural policies can contain reusable behavioral abstractions that differentiable decomposition can expose to downstream learners.

While we make important progress in extracting reusable subprograms from entangled neural representations, a few limitations remain. DIDEC assumes access to trained policies with compatible observation and action spaces. DIDEC also requires offline computation before downstream learning. In MiniGrid, learning and selecting options takes about 47 minutes on 16 CPU cores, compared with about one minute for Dec-Whole (Appendix I). This cost is not reflected in our learning curves, which only count environment interactions in the downstream task. The continuous-control variant does not yet instantiate temporally extended options. While we showed how the decomposition can work with both binary and continuous features, more research is needed to handle richer inputs, such as pixels. Similarly, input masks either read each coordinate from the environment or replace it with a default value, so they cannot represent transformations that shift input values or couple multiple input dimensions. Extending DIDEC to such structured transformations is an interesting direction for future work, as is studying online decomposition schemes to discover options while learning within a task.

8 Acknowledgements

This research was supported by Canada's NSERC and the CIFAR AI Chairs program. This research was enabled in part by support provided by the Digital Research Alliance of Canada. The authors thank Mahdi Alikhasi for helping out with an early implementation of the Dec-Options, and the anonymous reviewers for valuable feedback on this work.

References

Mahdi Alikhasi and Levi Lelis. Unveiling options with neural network decomposition. In *International Conference on Learning Representations*, 2024.

Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *AAAI conference on artificial intelligence*, volume 31, 2017.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.

Eseoghene Ben-Iwhiwhu, Saptarshi Nath, Praveen K Pilly, Soheil Kolouri, and Andrea Soltoggio. Lifelong reinforcement learning with modulating masks. *arXiv preprint arXiv:2212.11110*, 2022.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

David Bertoin, Adil Zouitine, Mehdi Zouitine, and Emmanuel Rachelson. Look where you look! Saliency-guided q-networks for generalization in visual reinforcement learning. In *Neural Information Processing Systems*, 2022.

Matthew Bowers, Theo X. Olausson, Lionel Wong, Gabriel Grand, Joshua B. Tenenbaum, Kevin Ellis, and Armando Solar-Lezama. Top-down synthesis for library learning. *Proceedings of the ACM on Programming Languages*, 2023.

David Cao, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. Babble: Learning better abstractions with e-graphs and anti-unification. *Proceedings of the ACM on Programming Languages*, 2023.

Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *CoRR*, abs/2306.13831, 2023.

Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling, 2014.

Alexander Clegg, Wenhao Yu, Zackory Erickson, Jie Tan, C Karen Liu, and Greg Turk. Learning to navigate cloth using haptics. In *Intelligent Robots and Systems*, pages 2799–2805, 2017.

Will Dabney, Georg Ostrovski, and Andre Barreto. Temporally-extended ε-greedy exploration. In *International Conference on Learning Representations*, 2021.

Kevin Ellis, Lionel Wong, Maxwell Nye, Mathias Sabl-Meyer, Luc Cary, Lore Pozo, Luke Hewitt, Armando Solar-Lezama, and Joshua Tenenbaum. Dreamcoder: growing generalizable, interpretable knowledge with wake-sleep bayesian program learning. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 381, 06 2023. doi: 10.1098/rsta.2022.0050.

Kevin Frans, Jonathan Ho, Xi Chen, Pieter Abbeel, and John Schulman. Meta learning shared hierarchies. *arXiv preprint arXiv:1710.09767*, 2017.

Bram Grooten, Tristan Tomilin, Gautham Vasan, Matthew E. Taylor, A. Rupam Mahmood, Meng Fang, Mykola Pechenizkiy, and Decebal Constantin Mocanu. Madi: Learning to mask distractions for generalization in visual deep reinforcement learning. In *International Conference on Autonomous Agents and Multiagent Systems*, pages 733–742, 2024.

Geoffrey E. Hinton, Peter Dayan, Brendan J. Frey, and Radford M. Neal. The "wake–sleep" algorithm for unsupervised neural networks. *Science*, 268(5214):1158–1161, 1995. doi: 10.1126/science.7761831.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.

Maximilian Igl, Andrew Gambardella, Jinke He, Nantas Nardelli, N Siddharth, Wendelin Böhmer, and Shimon Whiteson. Multitask soft option learning. In *Conference on Uncertainty in Artificial Intelligence*, pages 969–978, 2020.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

Yuu Jinnai, Jee W. Park, Marlos C. Machado, and George Konidaris. Exploration in reinforcement learning with deep covering options. In *International Conference on Learning Representations*, 2020.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. In *Proceedings of the National Academy of Sciences*, volume 114, pages 3521–3526. National Acad Sciences, 2017.

Martin Klissarov and Marlos C. Machado. Deep laplacian-based options for temporally-extended exploration. In *International Conference on Machine Learning*. JMLR.org, 2023.

George Konidaris and Andrew Barto. Building portable options: Skill transfer in reinforcement learning. In *International Joint Conference on Artificial Intelligence*, pages 895–900, 2007.

Anurag Koul, Alan Fern, and Sam Greydanus. Learning finite state representations of recurrent policy networks. In *International Conference on Learning Representations*, 2019.

David Kuric and Herke van Hoof. Reusable options through gradient-based meta learning. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL `https://openreview.net/forum?id=qdDmxzGuzu`.

Heinrich Küttler, Nantas Nardelli, Alexander H. Miller, Roberta Raileanu, Matteo Selvatici, Edward Grefenstette, and Tim Rocktäschel. The Nethack learning environment. In *Conference on Neural Information Processing Systems*, 2020.

Yiren Lu, Justin Fu, George Tucker, Xinlei Pan, Eli Bronstein, Rebecca Roelofs, Benjamin Sapp, Brandyn White, Aleksandra Faust, Shimon Whiteson, et al. Imitation is not enough: Robustifying imitation with reinforcement learning for challenging driving scenarios. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7553–7560. IEEE, 2023.

Marlos C. Machado, André Barreto, Doina Precup, and Michael Bowling. Temporal abstraction in reinforcement learning with the successor representation. *Journal of Machine Learning Research*, 24:1–69, 2023.

Timothy A. Mann and Shie Mannor. Scaling up approximate value iteration with options: Better policies with fewer iterations. In *International Conference on Machine Learning*, pages 127–135, 2014.

Volodymyr Mnih, Adrià-Puigpelat Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous Methods for Deep Reinforcement Learning. In *International Conference on Machine Learning*, 2016.

Guido Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Neural Information Processing Systems*, page 2924–2932, 2014.

Ashvin Nair, Bob McGrew, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Overcoming exploration in reinforcement learning with demonstrations. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, page 6292–6299. IEEE Press, 2018. doi: 10.1109/ICRA.2018.8463162. URL https://doi.org/10.1109/ICRA.2018.8463162.

Laurent Orseau, Levi H. S. Lelis, Tor Lattimore, and Théophane Weber. Single-agent policy tree search with guarantees. In *Neural Information Processing Systems*, page 3205–3215, 2018.

Alessandro B. Palmarini, Christopher G. Lucas, and N. Siddharth. Bayesian program learning by decompiling amortized knowledge. In *International Conference on Machine Learning*, 2024.

Silviu Pitis. Beyond binary: Ternary and one-hot neurons. https://r2rt.com/beyond-binary-ternary-and-one-hot-neurons.html, 2017. Accessed: 2025-05-12.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

Habibur Rahman, Thirupathi Reddy Emireddy, Kenneth Tjhia, Elham Parhizkar, and Levi Lelis. Synthesizing libraries of programs with auxiliary functions. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.

David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In *Neural Information Processing Systems*, volume 32, 2019.

Andrei A. Rusu, Sergio Gomez Colmenarejo, Caglar Gulcehre, Guillaume Desjardins, James Kirkpatrick, Razvan Pascanu, Volodymyr Mnih, Koray Kavukcuoglu, and Raia Hadsell. Policy distillation, 2016a. URL https://arxiv.org/abs/1511.06295.

Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016b.

Mikayel Samvelyan, Robert Kirk, Vitaly Kurin, Jack Parker-Holder, Minqi Jiang, Eric Hambro, Fabio Petroni, Heinrich Kuttler, Edward Grefenstette, and Tim Rocktäschel. Minihack the planet: A sandbox for open-ended reinforcement learning research. In *NeurIPS 2021 Datasets and Benchmarks Track*, 2021.

Stefan Schaal. Learning from demonstration. In *Neural Information Processing Systems*, 1997.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Jonathan Schwarz, Wojciech Czarnecki, Jelena Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. Progress & compress: A scalable framework for continual learning. In *International conference on machine learning*, pages 4528–4537, 2018.

Kun Shao, Yuanheng Zhu, and Dongbin Zhao. Starcraft micromanagement with reinforcement learning and curriculum transfer learning. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 3(1):73–84, 2018.

Avi Singh, Huihan Liu, Gaoyue Zhou, Albert Yu, Nicholas Rhinehart, and Sergey Levine. Parrot: Data-driven behavioral priors for reinforcement learning. *arXiv preprint arXiv:2011.10024*, 2020.

Richard S Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2):181–211, 1999.

Chen Tessler, Shahar Givony, Tom Zahavy, Daniel Mankowitz, and Shie Mannor. A deep hierarchical approach to lifelong learning in minecraft. In *AAAI conference on artificial intelligence*, volume 31, 2017.

Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. Supermasks in superposition. In *Neural Information Processing Systems*, volume 33, pages 15173–15184, 2020.

DIDEC					Fine-Tuned Policy				
0.6	0.6		0.6	0.6	0.0	0.0		0.0	0.0
0.6	0.6	0.6	0.6	0.6	0.0	0.0	0.2	0.0	0.0
	0.6	0.6	0.6			0.0	0.0	0.0	
0.6	0.6	0.6	1.0	0.6	0.0	0.0	0.0	0.0	0.0
0.6	0.6		0.6	0.6	0.0	0.0		0.0	0.0

Table 1: Generalization results of options generated by DIDEC and Fine-Tuned Policy in ComboGrid.

Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. *arXiv preprint arXiv:1708.01547*, 2017.

Tao Yu, Zhizheng Zhang, Cuiling Lan, Yan Lu, and Zhibo Chen. Mask-based latent reconstruction for reinforcement learning. In *Neural Information Processing Systems*, 2022.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Avnish Narayan, Hayden Shively, Adithya Bellathur, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning, 2021.

A Societal Impact

This paper advances basic research in sequential decision making. Our experiments are conducted in controlled benchmark domains, and we do not expect the specific systems evaluated here to have immediate societal impact. In the long run, however, methods for discovering reusable behavioral abstractions could have dual-use implications, since they may be applied to sequential decision-making systems more broadly. Potential benefits include improving sample efficiency and transfer in autonomous agents, while potential risks include enabling more capable autonomous behavior in settings where deployment is unsafe or insufficiently supervised.

B Examples of Options Learned in ComboGrid

Figure 4 shows the options learned in ComboGrid. In this domain, the learned options are modular and encode well-defined behaviors in the underlying LSTM, such as moving forward twice and turning right. The agent learned options that are independent of the observation, which is helpful in this domain, since the action combination does not change based on the observation. The agent can solve the test problem using only options.

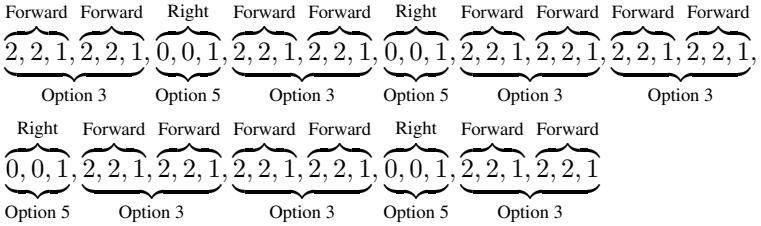


Figure 4: Sequence of actions DIDEC's agent learned on ComboGrid. Labels at the top describe the semantics of a combo, while those at the bottom describe the options learned.

Each cell in Table 1 shows the fraction of DIDEC's (top) and Fine-Tuned Policy's (bottom) options that generalize to the unseen observations of the test problem; the gray cells denote goal locations, so we did not calculate their values. At least 60% of DIDEC's options, which encode helpful combos such as 'move, move', generalize, while Fine-Tune's options fail. Recall that Fine-Tune differs from DIDEC only in the weights it adapts—Fine-Tune adjusts model weights, whereas DIDEC learns masks. This result illustrates how extracting subprograms with default parameters can enable generalization.

C Varying the Training Environments

Random training instances. The training problems used in our main experiments were chosen arbitrarily, which raises the question of whether DIDEC's advantage depends on a specific choice. To check this, we re-ran MultiRoom with three randomly selected training problems and evaluated on the test problems with seeds 230 and 431. Figure 5 shows the result: DIDEC's sample efficiency advantage over the baselines is similar to what we report in Figure 2.

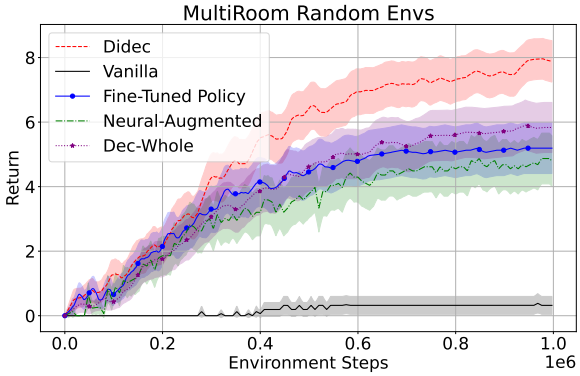


Figure 5: Learning curves on MultiRoom test problems 230 and 431 when the three training problems are selected randomly.

Size of the training set. The quality of the options DIDEC learns depends on the problems $\{P_j\}_{j=1}^{i-1}$ used for training. Because the masking objective extracts a subprogram of one source policy that reproduces a sub-trajectory generated by a different source policy, a behavior must appear in at least two training problems for DIDEC to extract a generalizing option for it. Adding a problem whose behaviors do not appear elsewhere enlarges the candidate pool without adding new reusable options.

ComboGrid makes this concrete. Figure 6 compares DIDEC with two, three, and four training problems. With two problems, the trajectories share the move-forward combo $(2, 2, 1)$ and the turn-right combo $(0, 0, 1)$, and DIDEC extracts options for both. The third problem adds the turn-left combo $(1, 1, 0)$, but it appears in only one trajectory, so no generalizing option is extracted for it; the candidate pool grows from $|\Omega| \approx 150$ to ≈ 450 without adding reusable behaviors. The fourth problem also contains $(1, 1, 0)$, completing coverage of all three combos and yielding the best average sample efficiency.

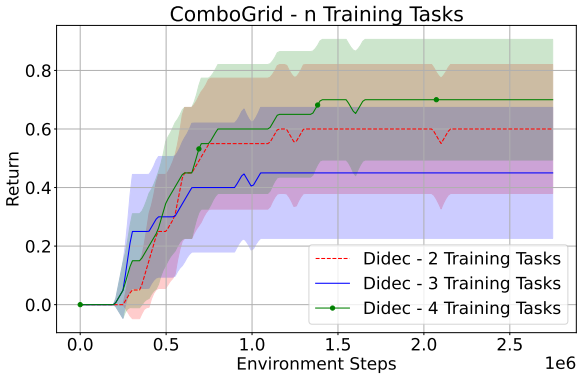


Figure 6: DIDEC's performance on ComboGrid (sparse reward) with two, three, and four training problems. Curves show average return with 95% confidence intervals over 20 seeds.

The drop at three tasks also reflects a selection effect: the larger candidate pool makes the local-search subset selection less reliable under a fixed budget of restarts. We observed the same pattern in MultiRoom, when increasing the training set from three to six problems, where the smaller set

$z_{\max}$	Seeds	Return (95% CI)	AUC (95% CI)
6	10	0.70 [0.35, 1.00]	0.61 [0.31, 0.92]
9	10	1.00 [1.00, 1.00]	0.82 [0.74, 0.89]

Table 2: Effect of $z_{\max}$ on DIDEC in ComboGrid with sparse reward. Return is the average return at the end of training (6M steps); AUC is the average return over training. Returns range from 0 to 1.

already covered the behaviors needed at test time. Increasing the search budget can mitigate this, and improving the scalability of the selection procedure is a natural direction for future work.

Effect of $z_{\max}$. The hyperparameter $z_{\max}$ controls the length of the longest sub-trajectories used for learning masks (Section 4.4). Longer sub-trajectories generate longer options, but they also tend to be more specialized to the source task. To evaluate this trade-off, we ran DIDEC on ComboGrid with sparse reward with $z_{\max} = 6$ and $z_{\max} = 9$ (Table 17). Table 2 shows the results after 6M steps. While there is a difference between the two values, DIDEC outperforms all other baselines with both $z_{\max} = 6$ and $z_{\max} = 9$. With $z_{\max} = 9$, DIDEC solves the test problem in all 10 seeds. This is the configuration we report in Table 3.

D Comparison with Other Families of Baselines

Alikhasi and Lelis [2024] showed that the enumerative version of DIDEC outperforms option-learning and transfer methods that were not designed to leverage policies trained on previous tasks. In this section, we compare DIDEC with methods from other families that use the training problems. **FAMP** meta-learns options over the training problems. **Distillation** follows DIDEC's pipeline, but trains a network with 16 hidden units on each sub-trajectory instead of learning masks; 16 is close to the effective size of DIDEC's masked models (Appendix G). **Behavior Prior** trains PPO with a KL-divergence penalty toward the average of the policies trained on the training problems. We swept the hyperparameters of these baselines over three seeds, as described in Appendix F, and ran the best configuration with 14 seeds in MultiRoom and 10 seeds in ComboGrid.

Domain	Method	Seeds	Return (95% CI)	AUC (95% CI)
MultiRoom (230)	DIDEC	14	8.21 [6.06, 10.00]	4.66 [3.08, 6.18]
	FAMP	14	0.15 [0.00, 0.38]	0.06 [0.01, 0.13]
	Distillation	14	0.00 [0.00, 0.00]	0.01 [0.00, 0.02]
	Behavior Prior	14	8.57 [6.43, 10.00]	6.96 [5.27, 8.24]
ComboGrid (sparse)	DIDEC	10	1.00 [1.00, 1.00]	0.82 [0.74, 0.89]
	Behavior Prior	10	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]

Table 3: Comparison of DIDEC with baselines from other families. Return is the average return at the end of training (1M steps in MultiRoom and 6M steps in ComboGrid); AUC is the average return over training. Returns range from 0 to 10 in MultiRoom and from 0 to 1 in ComboGrid (sparse). FAMP and Distillation were evaluated on MultiRoom only.

Table 3 shows the results on MultiRoom (230) and ComboGrid (sparse). FAMP and Distillation do not solve the test problem within 1M steps. A small network approximates the sub-trajectories well, but it does not carry the bias needed to generalize to problems with different structure. Since Distillation differs from DIDEC only in the model trained on each sub-trajectory, this result suggests that searching for subprograms of trained policies, rather than fitting new networks, is what makes the options reusable. We conjecture that FAMP fails for a similar reason: its options are meta-learned on the training problems, but do not transfer to the change in room structure from Unlock to MultiRoom. Behavior Prior learns faster than DIDEC (AUC of 6.96 versus 4.66), and both reach similar returns by the end of training. Behavior prior performs well here because the optimal behavior is similar in training and test problems, and the egocentric view keeps observations familiar.

This is not the case in ComboGrid (sparse), where the observation is not egocentric and the prior has no data from the cells the agent visits in the test problem. After 6M steps, Behavior Prior obtains a return of 0.00 in all 10 seeds, while DIDEC solves the test problem in all of them.

E Neural Architectures

We adopt a neural Actor-Critic framework. The model is composed of two networks with shared input: an actor that parametrizes a stochastic policy, and a critic that estimates the value function.

Let d_{obs} denote the dimensionality of the observation vector (after flattening), and let $|\mathcal{A}|$ be the number of discrete actions.

E.1 MiniGrid and MiniHack Network Architectures

The following network was used in FourRooms and MultiRoom. The architecture used with MiniHack is identical, except that it uses 256 neurons instead of 64.

Actor Network. The actor network maps an input observation to a distribution over actions via the following architecture:

- A linear layer with input size d_{obs} and output size 64,
- ReLU activation,
- A final linear layer with output size $|\mathcal{A}|$ producing action logits.

The final layer is initialized with a reduced standard deviation (std $= 0.01$) to promote stability during early exploration.

Critic Network. The critic network has a deeper architecture to estimate the state value:

- A linear layer from d_{obs} to 64 units,
- ReLU activation,
- Another linear layer with 64 hidden units,
- ReLU activation,
- A final linear layer mapping to a scalar value.

Weight Initialization. Weights are initialized orthogonally with a gain factor of $\sqrt{2}$, and biases are set to 0.0. The final layer of the critic is initialized with standard deviation of 1.0.

E.2 Meta-World Network Architecture

For Meta-World we use the SB3 PPO implementation with its default `MlpPolicy`. The actor and critic are separate networks that share no layers.

Actor Network. The actor network maps an input observation to a diagonal Gaussian distribution over continuous actions via the following architecture:

- A linear layer with input size d_{obs} and output size 64,
- tanh activation,
- Another linear layer with 64 hidden units,
- tanh activation,
- A final linear layer with output size $|\mathcal{A}|$ producing the action mean.

The log standard deviation of the Gaussian is a learned, state-independent parameter vector of size $|\mathcal{A}|$, initialized to 0.0. The final layer is initialized with a reduced gain (0.01) to promote stability during early exploration.

Hyperparameter	Values Tested
Learning rate	{0.01, 0.005, 0.001, 0.0005, 0.00005}
Clipping coefficient	{0.05, 0.1, 0.15, 0.2, 0.3}
Entropy coefficient	{0.01, 0.02, 0.03, 0.05, 0.1, 0.2}
Training time steps	{1 million}

Table 4: Hyperparameters evaluated for DIDEC on FourRooms (5000).

Critic Network. The critic network has the same hidden structure as the actor to estimate the state value:

- A linear layer from d_{obs} to 64 units,
- tanh activation,
- Another linear layer with 64 hidden units,
- tanh activation,
- A final linear layer mapping to a scalar value.

Weight Initialization. Weights are initialized orthogonally with a gain factor of $\sqrt{2}$, and biases are set to 0.0. The final layer of the critic is initialized with a gain of 1.0.

E.3 LSTM Architecture

We use the Stable-Baselines3 (SB3) [Raffin et al., 2021] implementation of PPO with an LSTM representation. The policy consists of: (i) two unidirectional LSTMs, one for the actor and another for the critic, with 16 neurons that produce a latent state; and (ii) separate, feedforward network heads for actor and critic. This network uses a $\tanh$ activation function, so we use input masking only in our LSTM experiments.

Actor Network. From the last LSTM output at each timestep, the actor head applies:

- A linear layer with 32 units,
- $\tanh$ activation,
- A final linear layer with output size $|\mathcal{A}|$ producing action logits.

Critic Network. The critic has its own LSTM, followed by its own head:

- A linear layer with 32 units,
- $\tanh$ activation,
- A final linear layer mapping to a scalar value.

Weight Initialization. Weights are initialized orthogonally with a gain factor of $\sqrt{2}$, and biases are set to 0.0. The final actor layer is initialized with a smaller weight scale of 0.01. The final critic layer is initialized with a gain of 1.0.

F Hyperparameter Sweeps

The hyperparameter sweeps were performed with a grid search over the values shown in the tables of this section. We considered three seeds for each set of values, and the value of a set is the average area under the curve (AUC) of the agent's return across the three seeds. We report the result of each approach using the hyperparameter set that has the largest average AUC. We consider the $s_{\max}$ of 5 and 10 across different benchmarks. Experiments were performed on Intel Xeon Gold 6448Y CPUs.

Tables 4 and 5 present the sweep results for FourRooms, while Tables 6, 7, and 8 present the sweep results for MultiRoom. Tables 19 and 20 present the sweep results for Distillation, Behavior Prior,

Methods	Learning rate	Clipping coefficient	Entropy coefficient
Didec	0.005	0.3	0.02
Fine-Tune	0.005	0.3	0.03
Dec-Whole	0.001	0.2	0.03
Neural-Augmented	0.001	0.3	0.03
Vanilla	0.0005	0.3	0.03

Table 5: Chosen hyperparameters per method on FourRooms (5000).

Hyperparameter	Values Tested
Learning rate	{0.01, 0.005, 0.001, 0.0005, 0.00005}
Clipping coefficient	{0.01, 0.05, 0.1, 0.15, 0.2, 0.3}
Entropy coefficient	{0.0, 0.01, 0.02, 0.03, 0.05, 0.1, 0.2}
Training time steps	{1 million}

Table 6: Hyperparameters evaluated for DIDEC on MultiRoom.

and FAMP on MultiRoom (230), and Tables 17 and 18 present the sweep results for DIDEC and Behavior Prior on ComboGrid. To keep the sweep affordable, Behavior Prior was trained for 200 thousand time steps and FAMP was meta-trained for 150 epochs during the sweep; the final runs use longer budgets. In some cases, the value selected in the sweep is a "border value", i.e., the smallest or the largest value attempted in the sweep. While methods using border values could perform better by increasing the size of our sweep, this experimental bias is present in all evaluations, not just the baselines. This means that DIDEC's performance could also be improved in some cases.

G Selecting Masking Type

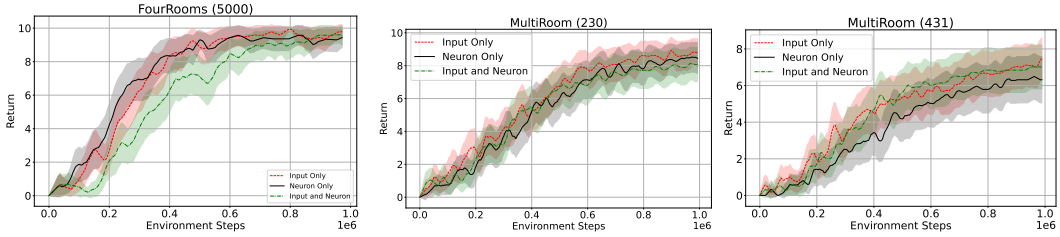


Figure 7: Evaluating masking schemes: input only, neurons only, and both. The plots present the average return and the 95% confidence intervals for 50 (FourRooms) and 57 seeds (MultiRoom).

We considered masking neurons, input features, or both as a hyperparameter. The results presented in the paper are based on the selection of the best-performing scheme for DIDEC. Figure 7 presents the performance of DIDEC when masking only the input features, only the neurons, and both. There is no noticeable difference among the different masking schemes in these three environments. This is not the case in ComboGrid with sparse reward, where the observation is not egocentric and changes more drastically from training to test. Table 21 shows that DIDEC with input masks solves the test problem in all 10 seeds, while DIDEC with neuron masks fails in all of them. After inspecting the failure cases, we observed that the options learned with neuron masks do not generalize to the test problem, whose structure differs from that of the training problems. This is the setting of Example 3: input masks allow the extracted computation to behave as if it were called in the context in which it was learned. The configuration with input masks is the one we report in Table 3.

Figure 8 shows the DIDEC results for LSTM policies on ComboGrid (dense), where one uses options learned only by learning the hidden state and memory vectors of the LSTM (LSTM Only), while the other learns the initial hidden state and memory vectors in addition to input masking (input and LSTM). These results highlight the need for both selecting the initial vectors and masking the inputs.

Methods	Learning rate	Clipping coefficient	Entropy coefficient
Didec	0.01	0.2	0.01
Fine-Tune	0.01	0.3	0.02
Dec-Whole	0.001	0.3	0.02
Neural-Augmented	0.01	0.15	0.02
Vanilla	0.001	0.3	0.01

Table 7: Chosen hyperparameters per method on MultiRoom (230).

Methods	Learning rate	Clipping coefficient	Entropy coefficient
Didec	0.001	0.2	0.03
Fine-Tune	0.01	0.2	0.02
Dec-Whole	0.001	0.3	0.01
Neural-Augmented	0.005	0.15	0.05
Vanilla	0.001	0.3	0.01

Table 8: Chosen hyperparameters per method on MultiRoom (431).

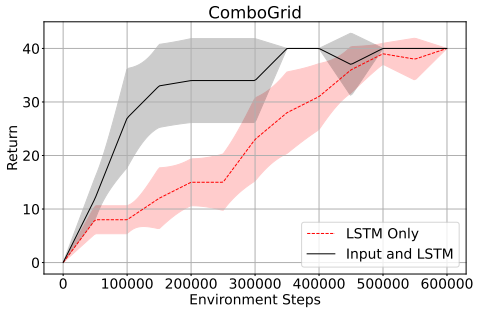


Figure 8: Comparison between learning only the initial hidden state and memory vectors of the LSTM (LSTM only), and learning the vectors and also masking the inputs.

A helpful analogy is that by learning the initial vectors, we select the initial mode of the LSTM's underlying FSM; by masking the inputs, we control which transitions we allow from that initial mode. By doing both, we are selecting a sub-automaton of the LSTM underlying FSM.

Note that, by choosing default values for the inputs, we select subtrees of the underlying neural tree, implicitly masking neurons. An advantage of masking neurons is that it works as a compression scheme. Every neuron that is set to either active or inactive can be removed from the network, thus compressing the model. Table 22 shows the effective size of the options DIDEC extracts in MultiRoom. Masking both inputs and neurons reduces the number of actor parameters by 85.3%, and the options retain about 20 of the 64 hidden units. We use this number to set the size of the Distillation baseline in Appendix D: a network with 16 hidden units, which has $13{,}126$ parameters.

H Exploration with DIDEC's Options

Figure 9 presents the heatmaps denoting the probability of an agent visiting each cell in the first 500,000 training steps of the Advantage Actor-Critic (A2C) [Mnih et al., 2016] agent, for Vanilla, Dec-Whole, Fine-Tune and DIDEC (from left to right). DIDEC allows for better exploration as the agent learns to navigate between rooms much more quickly than Vanilla, as evidenced by the lighter rectangle connecting the four rooms.

Hyperparameter	Values Tested
Learning rate	{0.01, 0.005, 0.001, 0.0001}
Clipping coefficient	{0.1, 0.2, 0.3}
Entropy coefficient	{0.0, 0.01, 0.02, 0.05}
Rollout Length	{128}
Training time steps	{1 million}

Table 9: Hyperparameters evaluated for DIDEC on ComboGrid - Sparse Reward.

Hyperparameter	Values Tested
Learning rate	{0.005, 0.001, 0.0001, 0.0005}
Clipping coefficient	{0.1, 0.2, 0.3}
Entropy coefficient	{0.0, 0.01, 0.02, 0.05}
Rollout Length	{128, 256}
Training time steps	{3 million}

Table 10: Hyperparameters evaluated for DIDEC on ComboGrid - Dense Reward.

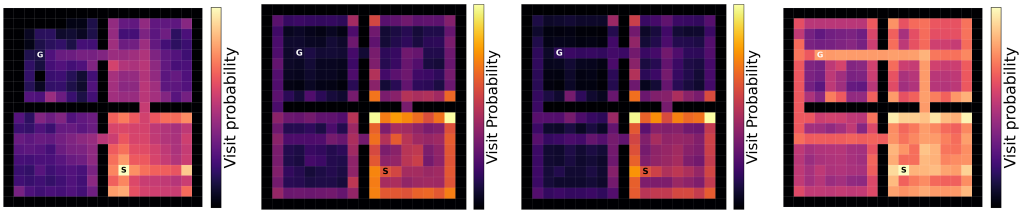


Figure 9: Probability of an agent visiting each of the cells in FourRooms in the first 500,000 training steps. From left to right Vanilla, Dec-Whole, Fine-Tune, DIDEC. The data is averaged over 50 seeds.

I Time Analysis of Option Extraction

To quantify the computational overhead introduced by option extraction and option selection, we measured wall-clock time for each system evaluated on MiniGrid SimpleCrossing and our Meta-World task under an identical compute setup. For MiniGrid, all methods run on a single CPU job with 16 physical CPU cores and 32 parallel worker processes (Python multiprocessing). For Meta-World we use a single CPU with a single process. No GPUs are used in these experiments.

For DIDEC, FineTune, and Dec-Whole we report the average and standard deviation of

1. Learning Time: time to learn all candidate options across the base problems.
2. Selection Time: time to select a subset that reduces the Levin loss.
3. Total Time: the sum of the two steps.

All results shown in Table 23. In MiniGrid, DIDEC's running time is comparable to that of Fine-Tune, but with a much improved sample efficiency (see Figure 2). DIDEC's running time of learning options is on par with the time the agent spends exploring the environment. This is loosely related to wake-sleep algorithms [Hinton et al., 1995]. In DIDEC, the agent goes through its wake cycle, where it learns to solve problems, and its sleep cycle, it organizes what it has learned into reusable options. The experiments in Meta-World show that DIDEC's neural decomposition can be used in much simpler settings, where the running time can be reduced from minutes to seconds.

J Subset Selection and Levin-Loss Computation

DIDEC can generate a large candidate set of options. Using all options in this set can slow down downstream learning because the high-level policy must learn to choose among a large number of

Methods	Learning rate	Clipping coefficient	Entropy coefficient
Didec	0.005	0.3	0.01
Fine-Tune	0.005	0.1	0.01
Neural-Augmented	0.005	0.1	0.01
Vanilla	0.005	0.1	0.02

Table 11: Chosen hyperparameters per method on ComboGrid - Dense Reward. The chosen rollout length for all the methods was 128.

Methods	Learning rate	Clipping coefficient	Entropy coefficient
Didec	0.001	0.1	0.01
Fine-Tune	0.001	0.2	0.02
Neural-Augmented	0.001	0.2	0.02
Vanilla	0.001	0.2	0.02

Table 12: Chosen hyperparameters per method on ComboGrid - Sparse Reward.

temporally extended actions. We therefore prune the candidate set using the subset-selection criterion of Alikhasi and Lelis [2024]. This criterion is based on the Levin loss [Orseau et al., 2018], which estimates how many samples a policy would need to reproduce a set of trajectories.

Let Ω be the set of candidate options and let $\Omega' \subseteq \Omega$ be a candidate subset. For a trajectory

$$T = ((o_0, a_0), (o_1, a_1), \ldots, (o_{H-1}, a_{H-1}), o_H),$$

the Levin loss of a policy π on T is

$$L(T, \pi) = \frac{H}{\prod_{t=0}^{H-1} \pi(a_t \mid o_t)}.$$

The denominator is the probability that π reproduces the trajectory, and the numerator is the number of environment interactions required by one rollout. Thus, the loss estimates the number of environment interactions required before the policy samples the trajectory.

To simulate the beginning of learning, we evaluate subsets using the uniform policy over the option-augmented action space. Let $\pi_u^{\Omega'}$ denote the uniform policy over primitive actions and options in Ω'. Adding options increases the size of the action space, which decreases the probability of each high-level choice under the uniform policy. However, useful options can cover multiple primitive actions with a single high-level decision, reducing the effective trajectory length. Levin loss balances these two effects.

When computing the loss on a trajectory T_j generated by policy π_j, we follow prior work and do not allow options extracted from π_j to be used in the computation. This prevents the selection procedure from choosing options that reproduce the policy that generated the trajectory, which is unlikely to generalize to downstream tasks. Let Ω'_{-j} denote the subset of options in Ω' whose source policy is not π_j. The score of a subset is

$$\text{Score}(\Omega') = \sum_j L(T_j, \pi_u^{\Omega'_{-j}}).$$

Computing the Levin loss. The main step in subset selection is computing the Levin loss of a trajectory under a fixed option set. This requires deciding which options should be used at which points of the trajectory. Algorithm 2 gives the dynamic-programming procedure used by Dec-Options. The table M stores the minimum number of high-level decisions needed to reproduce each prefix of the trajectory. A primitive action advances the trajectory by one step, while an applicable option advances it by the duration of the option.

An option $\omega = \text{repeat}(z_\omega) : \pi_\omega$ is applicable at time t if it reproduces the next z_ω actions in the trajectory, that is, if the actions selected by π_ω on $o_t, o_{t+1}, \ldots, o_{t+z_\omega-1}$ match $a_t, a_{t+1}, \ldots, a_{t+z_\omega-1}$.

Hyperparameter	Values Tested
Actor Learning rate	{0.0001, 0.0003, 0.0005}
Critic Learning rate	{0.0001, 0.0003, 0.0005}
Clipping Coefficient	{0.1, 0.2}
Entropy Coefficient	{0.0, 0.01,0.02, 0.05}
Training time steps	{1 million}

Table 13: Hyperparameters evaluated for DIDEC on MiniHack (30).

Methods	Actor learning rate	Critic learning rate	Entropy coefficient
Didec	0.0001	0.0005	0.05
Fine-Tune	0.0005	0.0003	0.02
Dec-Whole	0.0001	0.0005	0.02
Neural-Augmented	0.0003	0.0005	0.02
Vanilla	0.0005	0.0005	0.02

Table 14: Chosen hyperparameters per method on MiniHack (30).

Greedy subset selection. The original Dec-Options procedure uses a greedy algorithm to select a subset of options. Starting with $\Omega' = \emptyset$, the algorithm repeatedly adds the option $\omega \in \Omega \setminus \Omega'$ that most decreases $\mathrm{Score}(\Omega' \cup \{\omega\})$. The process stops when no remaining option decreases the score. Algorithm 3 summarizes the procedure.

Stochastic hill climbing. In our experiments, we use the same Levin-loss objective, but approximate the subset-selection problem with stochastic hill climbing. A candidate solution is a subset $\Omega' \subseteq \Omega$ with $|\Omega'| \leq s_{\max}$, where $s_{\max}$ is a hyperparameter. The search starts from a randomly selected candidate subset and repeatedly moves to the best neighbor with smaller Levin loss. If no neighbor improves the loss, the search terminates. We use random restarts and return the candidate with the smallest loss across all restarts.

The neighborhood function samples v options from $\Omega \setminus \Omega'$. If $|\Omega'| < s_{\max}$, the neighborhood includes candidates obtained by adding one sampled option to Ω'. It also includes candidates obtained by replacing one option in Ω' with one sampled option, and candidates obtained by removing one option from Ω'. To bias the search toward complementary options, the v options are sampled according to how often they can be initiated at trajectory positions not already covered by the current subset. A trajectory position is uncovered if, in Algorithm 2, the dynamic program reaches the next state through a primitive action rather than through an option. This neighborhood therefore favors options that cover parts of the training trajectories not yet covered by the current subset.

K Problem Domains

We train one policy per training problem and roll out each policy once, so DIDEC uses a single trajectory per training problem. Overall, DIDEC extracts options from a small amount of data. We assume that every policy in Π_{train} solves its training problem. This assumption is reasonable in our experiments because the training problems are considerably easier than the test problems; for example, we train on Unlock and evaluate on MultiRoom, which is larger and requires navigating through several rooms. Our research question is whether we can extract useful subfunctions from trained neural networks. This question is only meaningful for networks that encode meaningful behavior, as we should not expect poorly trained models to contain helpful behaviors.

MiniGrid. We consider ten Simple-Crossing configurations as training problems and one Four-Rooms configuration as the test problem. We also use three Unlock environments for training and two MultiRoom configurations for testing. MultiRoom is more challenging than FourRooms because the agent must find a key and unlock a door, and because room shapes vary, making generalization harder. For SimpleCrossing and FourRooms we use three actions (turn left, turn right, move forward). For Unlock and MultiRoom, which require picking up a key and opening a door, we use six actions

Hyperparameter	Values Tested
Learning rate	{ 0.0008, 0.0003, 0.0001, 0.00005}
Entropy coefficient	{0.0, 0.001, 0.005}
Training time steps	{200000}

Table 15: Hyperparameters evaluated for baselines on Meta-World.

Methods	Learning rate	Entropy coefficient
Didec	0.0003	0.001
Fine-Tune	0.0003	0.0
Dec-Whole	0.00005	0.001
Neural-Augmented	0.00005	0.001
Vanilla	0.0001	0.005

Table 16: Chosen hyperparameters per method on Meta-World.

(turn left, turn right, move forward, pick up, drop, toggle). For all MiniGrid experiments we use egocentric 9×9 observation window.

MiniHack. We consider the corridors challenge of MiniHack, where the agent learns to navigate in long procedurally generated rooms and corridors of the game NetHack [Küttler et al., 2020]. This environment offers dynamics and challenges different from MiniGrid, such as the agent starving. We consider three environments as training, and a fourth environment with longer corridors and more rooms as the test problem. As in MiniGrid, we also use an egocentric view of size 9×9 around the agent.

ComboGrid. In ComboGrid, the agent must learn action sequences whose effects occur only after completing an "action combo". We consider three primitive actions (0, 1, 2) and three combos: 0,0,1 (turn right); 1,1,0 (turn left); and 2,2,1 (move forward). Because observations change only after a combo, the agent must remember sequences. Training uses four 5×5 grids surrounded by walls, leaving a 3×3 area where the agent collects a marker. For testing, we remove the walls to yield a full 5×5 grid with four markers. We study two reward settings: dense, where the agent receives a positive reward per marker, and sparse, which rewards only after collecting all markers.

Meta-World. In Meta-World, the agent controls a robotic arm to perform various tasks. As observations, the agent receives the coordinates of different objects in 3D space; as actions, it can control the direction of the robotic arm's gripper, as well as opening and closing it. Both observations and actions are continuous. In our experiments, we consider two training problems where the agent moves a plate into a hockey net. The training problems differ in the position of the arm, plate, and net. The test problems are more difficult as the distance between the plate and the net changes, one by 0.85 times the distance in the training problems (Closer Goal) and the other by 1.35 times the distance (Farther Goal). Since selecting a subset of options while minimizing the Levin loss is inherently discrete, we use a simplified version of DIDEC with Meta-World to handle its continuous spaces. When training masks, instead of considering all sub-trajectories of each $\mathcal{T}_j$ of $\mathcal{T}_{\text{train}}$, we consider only $\mathcal{T}_j$, thus resulting in one option per trajectory. Learned options are made available to the agent through a mixture-of-experts scheme [Jacobs et al., 1991] (Appendix L), thus bypassing the selection step. The experiments in Meta-World serve two purposes: to show that our mask-based decomposition can also be used in continuous spaces and in a simpler setting that bypasses the subset selection step.

Figure 10 shows the ten SimpleCrossing environments used in training, and the FourRooms environments used in testing. Figure 11 shows the training and test problems for MultiRoom. Interestingly, DIDEC outperforms the baselines by a large margin on problem 230 (Figure 2), whose middle room has a structure different than the training problems—the room is narrower than the rooms in problems 1, 3, and 17. This discrepancy can affect generalization because the agent has not previously observed this type of room. This result demonstrates the generalization capabilities of the learned options. For both FourRooms and MultiRoom, we use MiniGrid's standard reward function [Chevalier-Boisvert et al., 2023].

Hyperparameter	Values Tested
Learning rate	{0.005, 0.01}
Entropy coefficient	{0, 0.01, 0.05}
KL coefficient (Behavior Prior)	{0.03, 0.1, 0.3, 1.0}
Training time steps	{6 million}

Table 17: Hyperparameters evaluated for DIDEC and Behavior Prior on ComboGrid - Sparse Reward.

Hyperparameter	Chosen
Learning rate (DIDEC)	0.01
Entropy coefficient (DIDEC)	0.01
Mask type (DIDEC)	input
$z_{\max}$ (DIDEC)	9
Learning rate (Behavior Prior)	0.01
Entropy coefficient (Behavior Prior)	0.01
KL coefficient (Behavior Prior)	0.1

Table 18: Chosen hyperparameters for DIDEC and Behavior Prior on ComboGrid - Sparse Reward.

Figure 12 shows the four ComboGrid training problems and the test problem, where the agent needs to collect all four markers to complete the task. In the dense version of the environment, each marker gives a reward of ten; any other step gives zero reward. In the sparse version, the agent receives one after collecting all markers, and zero otherwise.

Figure 13 shows the training and test problems of the Corridors environment we used from MiniHack. In this environment, the agent receives a reward of -1 for every step, except for the transition to the stairs, when the episode terminates.

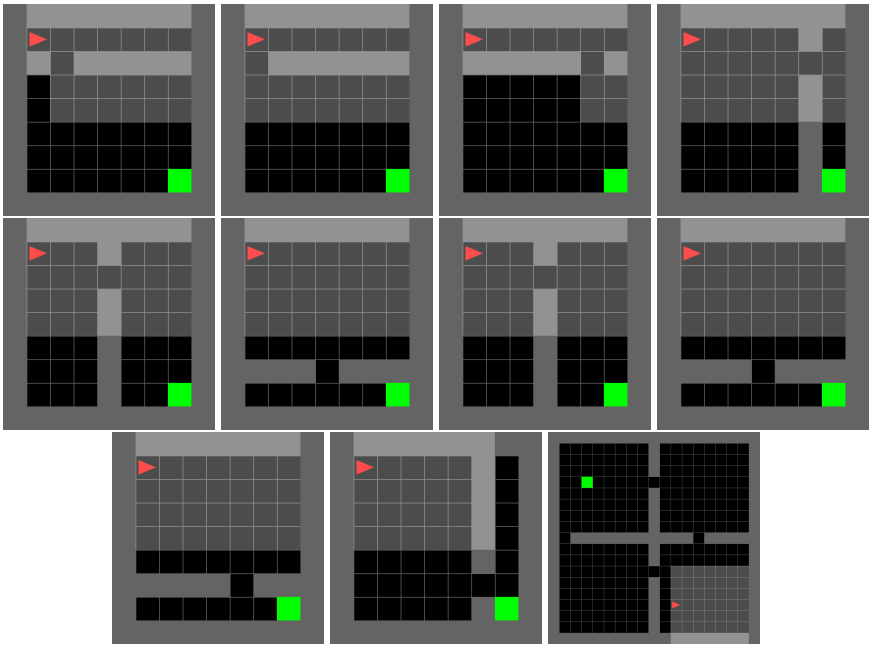


Figure 10: SimpleCrossing problems used as training for the FourRooms experiments. The seeds used to generate the problems, from left to right and top to bottom: 1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000. The bottom-right image shows the FourRooms environment used (5000).

Hyperparameter	Values Tested
Learning rate (Distillation)	{0.0003, 0.001, 0.003, 0.005, 0.01, 0.03}
Distillation rounds (Distillation)	{10, 20, 40}
Learning rate (Behavior Prior)	{0.0001, 0.00025, 0.001, 0.002, 0.005, 0.01}
Clipping coefficient (Behavior Prior)	{0.1, 0.2, 0.3}
Entropy coefficient (Behavior Prior)	{0.01, 0.05, 0.1, 0.15, 0.2, 0.3}
KL coefficient (Behavior Prior)	{0.01, 0.03, 0.05, 0.1, 0.2, 0.3, 0.5, 1, 10}
Training time steps (Behavior Prior)	{200 thousand}
Outer learning rate (FAMP)	{0.003, 0.01, 0.03}
Inner learning rate (FAMP)	{1, 5, 10}
Lookaheads (FAMP)	{1, 2, 3}
Number of options (FAMP)	{2, 4, 8}
Meta-training epochs (FAMP)	{150}

Table 19: Hyperparameters evaluated for the Distillation, Behavior Prior, and FAMP on MultiRoom (230). Sweeps were staged: an initial grid, then one-dimensional refinements around the best configuration.

Hyperparameter	Chosen
Learning rate (Distillation)	0.01
Distillation rounds (Distillation)	40
Learning rate (Behavior Prior)	0.005
Clipping coefficient (Behavior Prior)	0.2
Entropy coefficient (Behavior Prior)	0.1
KL coefficient (Behavior Prior)	0.1
Training time steps (Behavior Prior)	1 million
Outer learning rate (FAMP)	0.003
Inner learning rate (FAMP)	5
Lookaheads (FAMP)	1
Number of options (FAMP)	2
Meta-training epochs (FAMP)	300

Table 20: Chosen hyperparameters for Distillation, Behavior Prior, and FAMP on MultiRoom (230).

L Mixture of Experts for Continuous Action Spaces

To show a broader applicability of DIDEC's decomposition, we also apply it to a Meta-World task [Yu et al., 2021], which has continuous actions and state spaces. Instead of performing subset selection, for domains with continuous actions, we use a mixture-of-experts approach [Jacobs et al., 1991], combining the primitive actions with that of one or more options. The policy outputs an observation-dependent mean $\mu(o)$ and has a learned, state-independent log standard deviation $\log \sigma \in \mathbb{R}^{|A|}$ (Appendix E.2). Actions are sampled as

$$a = \mu(o) + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

where $\mu(o)$ is the output of the policy for o and $\sigma = \exp(\log \sigma)$ is shared across all observations.

Given a set of primitive and options of size K, the agent learns to mix them by learning K parameters θ.

$$a_{\text{mix}} = \sum_{i=1}^{K} \theta_i a_i, \qquad \theta_i \geq 0, \quad \sum_{i=1}^{K} \theta_i = 1\,.$$

Here, a_{mix} is the action the agent takes, and the mixture weights θ are produced with a Softmax network and learned end-to-end while the agent learns to solve a given POMDP.

Unlike options [Sutton et al., 1999], this mixture-of-experts approach does not produce temporally extended actions: each subprogram provides an action at every timestep.

Mask type	Seeds	Return (95% CI)	AUC (95% CI)
Input	10	1.00 [1.00, 1.00]	0.82 [0.74, 0.89]
Neuron	10	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]

Table 21: DIDEC with input masks and with neuron masks on ComboGrid with sparse reward. Return is the average return at the end of training (6M steps); AUC is the average return over training. Returns range from 0 to 1.

Masks	Options	Input masks (0 / 1 / read)	Neuron masks (inactive / active / retained)	Parameters	Reduction
Input	4.4 ± 0.7	$275.4/272.6/265.0$	– / – / 64	$17{,}416 \pm 401$	66.8%
Neuron	4.2 ± 0.6	– / – / 813	$19.3/24.7/20.0$	$21{,}285 \pm 1{,}472$	59.4%
Both	4.8 ± 0.4	$267.4/272.2/273.4$	$18.8/23.5/21.7$	$7{,}730 \pm 554$	85.3%

Table 22: Effective size of DIDEC's options in MultiRoom. The source policy has 813 binary inputs, 64 hidden units, and $52{,}486$ actor parameters. Input masks report the average number of inputs set to 0, set to 1, or read from the environment; neuron masks report the average number of hidden units masked as inactive, active, or retained.

MiniGrid			
Method	**Learning**	**Selection**	**Total**
DIDEC	1476.3 ± 760.4	1359.4 ± 631.1	2835.7 ± 1390.3
Fine-Tune	871.0 ± 464.9	1387.1 ± 668.8	2258.1 ± 1131.0
Dec-Whole	0.0 ± 0.0	53.4 ± 17.8	53.5 ± 17.8
Meta-World			
Method	**Learning**	**Selection**	**Total**
DIDEC	10.21 ± 0.05	-	10.21 ± 0.05
Fine-Tune	5.28 ± 0.02	-	5.28 ± 0.02
Dec-Whole	0.0 ± 0.0	-	0.0 ± 0.0

Table 23: Average running time in seconds and standard deviation for option learning and selection across different approaches. Dec-Whole does not perform any training, so the learning time is zero. The Meta-World experiments do not include a selection step because we use one option per base policy. We use 50 seeds for MiniGrid and 40 for Meta-World.

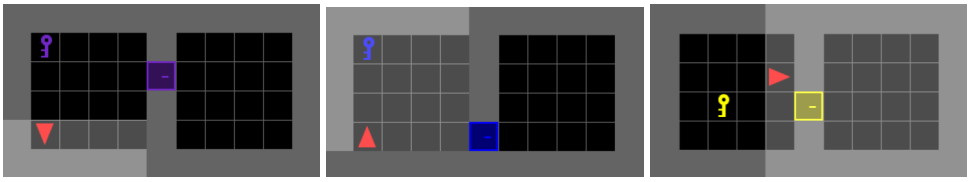
Algorithm 2 COMPUTE-LEVIN-LOSS

Require: Trajectory $T = ((o_0, a_0), \ldots, (o_{H-1}, a_{H-1}), o_H)$, option set Ω, primitive action set A
Ensure: $L(T, \pi_u^{\Omega})$
$p_{u,\Omega} \leftarrow 1/(|A| + |\Omega|)$
$M[j] \leftarrow j$ for $j = 0, 1, \ldots, H$
for $j = 0, 1, \ldots, H-1$ **do**
  $M[j+1] \leftarrow \min(M[j+1], M[j] + 1)$
  for each option $\omega \in \Omega$ **do**
    Let z_ω be the duration of ω
    if $j + z_\omega \leq H$ and ω is applicable at o_j **then**
      $M[j + z_\omega] \leftarrow \min(M[j + z_\omega], M[j] + 1)$
return $H \cdot (p_{u,\Omega})^{-M[H]}$

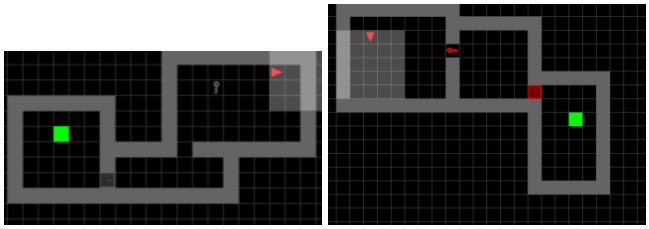
Algorithm 3 GREEDY-SUBSET-SELECTION

Require: Candidate options Ω, training trajectories $\mathcal{T}_{\text{train}}$, primitive action set A
Ensure: Selected subset Ω'

$\Omega' \leftarrow \emptyset$
$l \leftarrow \text{Score}(\Omega')$
while true **do**
  $\omega^\star \leftarrow \text{None}, l^\star \leftarrow l$
  for each option $\omega \in \Omega \setminus \Omega'$ **do**
    $c \leftarrow \Omega' \cup \{\omega\}$
    $l_c \leftarrow \text{Score}(c)$
    if $l_c < l^\star$ **then**
      $\omega^\star \leftarrow \omega, l^\star \leftarrow l_c$
  if $\omega^\star = \text{None}$ **then**
    break
  $\Omega' \leftarrow \Omega' \cup \{\omega^\star\}$
  $l \leftarrow l^\star$
return Ω'



(a) Training problems used in MultiRoom. The seeds used to generate the problems, from left to right: 1, 3, 17.



(b) Test problems used in MultiRoom. The seeds used to generate the problems, from left to right: 230 and 431.

Figure 11: Training and test problems used in our MultiRoom experiments.

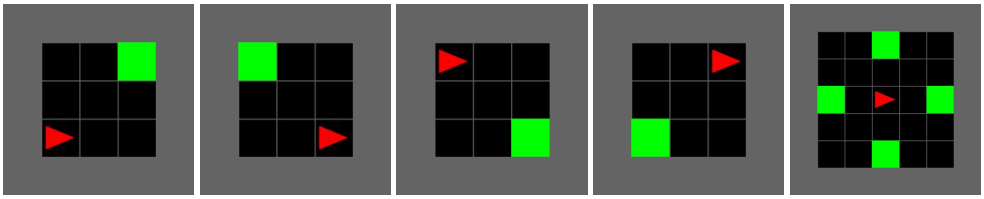


Figure 12: The four ComboGrid training environments, from left to right and top to bottom, and the test environment.

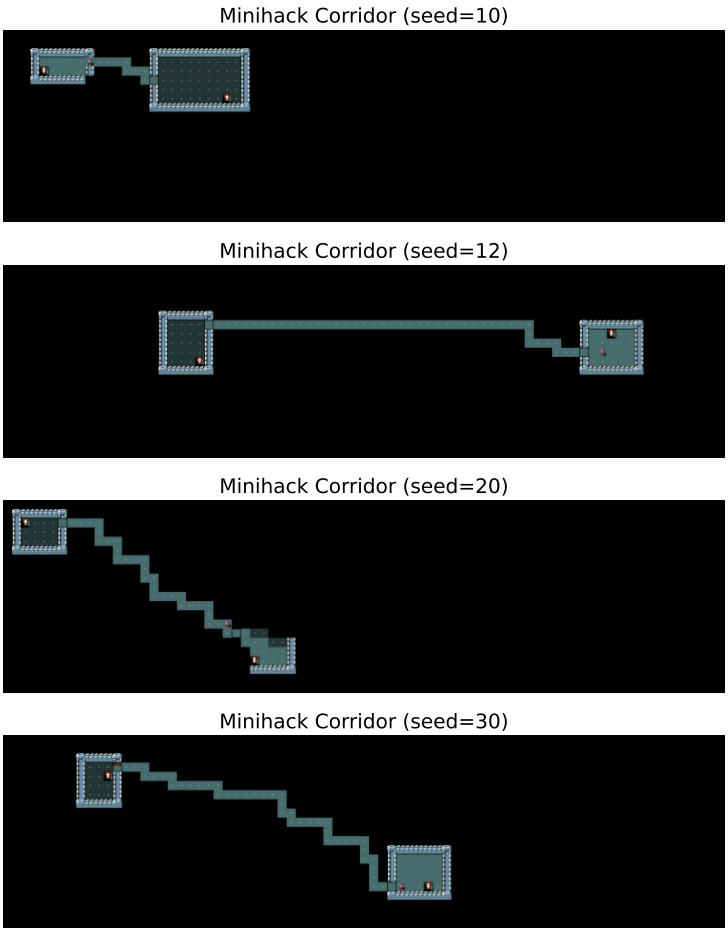


Figure 13: The three MiniHack Corridor training environments (top), and the test environment (bottom), generated with seeds 10, 12, 20, and 30, respectively.

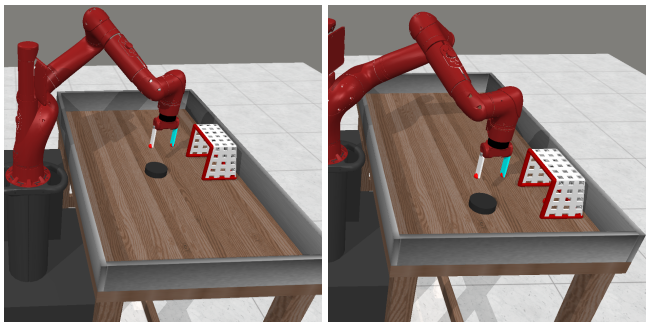


Figure 14: The two Meta-World training environments. In addition to the location of the net, the plate, and the arm, the distance between the plate and the net was changed in the test problems. The plate is closer to the goal in one test problem and farther in the other.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: All the claims are supported by either experiments or theoretical results.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: We include the limitations of the work in the last section of the paper.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: Prior to stating Theorem 1, we state all assumptions we need.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: We have an extensive appendix with all the information to reproduce the results. Even algorithms used from previous work is explained in the paper for completeness.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: Our submission is not accompanied by our implementation, although we will make the code available upon acceptance.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: To the best of our knowledge, we include all details required to reproduce the results.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: We used a sufficiently large number of seeds in our experiments and we report 95% confidence intervals.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: We provide the type of CPUs used in our experiments; we did not use GPUs in this work.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: We are familiar with the code of ethics and confirm that our research conforms with it.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: We have a section in the appendix, where we explain the broader implications of our work.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [No]

Justification: Our workd does not have safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We cite all benchmarks used in our paper.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.